

## Tip Control – Tip Kontrola

Ovo je roditeljska kontrola drugih kontrola (kao što su kontrole Form, Label i Button). Sve kontrole nasleđuju svojstva, metode i događaje definisane u ovom tipu, tako da ih možete koristiti sa svim kontrolama.

Na primer, možete koristiti svojstvo Name sa različitim kontrolama kao što je ova:

```
TextWindow.WriteLine(Me.Name)
TextWindow.WriteLine(Button1.Name)
TextWindow.WriteLine(Label1.Name)
TextWindow.WriteLine(TextBox1.Name)
```

### Animiranje kontrola

Klasa kontrole vam pruža mnoge metode animacije koje možete koristiti sa bilo kojom kontrolom za kreiranje lepih vizuelnih efekata. Ove metode su:

1. AnimateSize.
2. AnimatePos.
3. AnimateAngle.
4. AnimateColor.
5. AnimateTransparency.

Da biste videli ove metode u akciji, pogledajte projekte „Animation 1, 2 i 3“ u folderu sa primerima.

Važno je napomenuti da se u svim ovim metodama animacija izvršava asinhrono, tako da će vaša aplikacija nastaviti da radi i forma će reagovati na korisnika dok se kontrola animira. Takođe, linije koda pored linije koja je pozvala metod animacije biće izvršene odmah nakon početka animacije. Obratite pažnju na ove napomene:

- Možete animirati više kontrola istovremeno.
- Možete animirati različita svojstva iste kontrole istovremeno.
- Ne možete primeniti više od jedne animacije na isto svojstvo kontrole istovremeno, jer kada počnete da animirate svojstvo, zaustavlja se trenutna animacija ovog svojstva i počinje nova. Da biste rešili ovaj problem, možete odložiti izvršavanje koda dok se animacija ne završi, a zatim pokrenuti novu. Na primer:

```
Form1.AnimateColor(Colors.Red, 1000)
Program.Delay(1000)
Form1.AnimateColor(Colors.Yellow, 2000)
```

Gornji kôd će animirati boju pozadine forme kako bi je postepeno menjao iz trenutne u crvenu tokom jedne sekunde.

Izvršavanje koda je odloženo za jednu sekundu kako bi se osiguralo da je animacija završena, a zatim se pokreće druga animacija kako bi se postepeno promenila boja pozadine forme iz crvene u žutu za dve sekunde.

- Ne bi trebalo da menjate animaciju svojstva nakon pokretanja animacije, jer će to odmah prekinuti animaciju. Na primer:

```
Form1.AnimateColor(Colors.Red, 1000)
Form1.BackColor = Colors.White
```

Gornji kôd će odmah promeniti boju pozadine forme u belu i nećete videti nikakvu animaciju. Dakle, takođe treba da odložite izvršavanje druge komande dok se animacija ne završi:

```
Form1.AnimateColor(Colors.Red, 1000)
Program.Delay(1000)
Form1.BackColor = Colors.White
```

- Sada znate kako da zaustavite animaciju kad god želite ☺. Samo dodajte dugme za otkazivanje i u njegovom obrađivaču događaja `OnClick` promenite svojstvo koje se animira na bilo koju vrednost (pogledajte projekat „Otkazi animaciju“ u fascikli sa primerima).

- Iz gornjih napomena možete zaključiti da ako pokušate da pročitate animirano svojstvo odmah nakon pokretanja animacije, dobićete njegovu staru vrednost, stoga takođe morate da sačekate dok se animacija ne završi da biste dobili konačnu vrednost svojstva. Ali takođe možete koristiti tajmer da biste prikazali međuvrednosti tog svojstva dok se animira. Projekat „Otkazi animaciju“ u fascikli sa primerima vam pokazuje kako da to uradite.

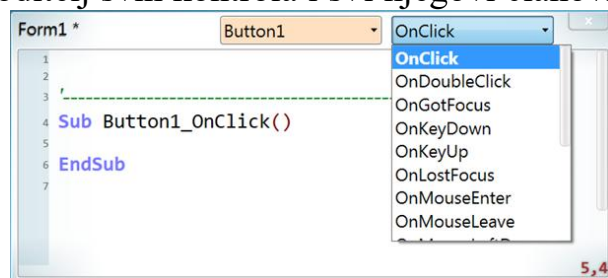
## Rukovanje kontrolnim događajima

Windows aplikacija ima grafički korisnički interfejs (GUI), koji se uglavnom sastoji od jedne ili više formi, sa kontrolama za primanje unosa od korisnika i prikazivanje rezultata. Korisnik unosi podatke mišem i tastaturom, tako da je neophodno da znamo šta radi pomoću ovih uređaja i kada. To možemo znati iz događaja.

Događaji su poruke koje Windows operativni sistem šalje programu da nas obavesti da korisnik trenutno nešto radi, kao što je klik na dugme, pritiskanje tastera ili pomeranje miša. Reagujemo na takve događaje izvršavanjem odgovarajućeg koda koji omogućava našoj aplikaciji da obavlja svoj posao. Ovo se naziva rukovanje događajima, a kôd koji reaguje na događaj naziva se rukovalac događajima, što je samo običan potprogram kojim kažemo sVB-u da pozove kada se događaj pokrene.

Dakle, Windows aplikacije su uglavnom gomila rukovalaca događajima koji čekaju da korisnik nešto uradi. Zato se moderno programiranje naziva programiranje vođeno događajima.

U sVB-u, svaka kontrola ima svoje događaje koji vas obaveštavaju o tome šta korisnik radi sa njom, kao što je događaj `OnTextChanged` koji se pokreće kada korisnik promeni sadržaj polja za tekst. Sve kontrole dele neke osnovne događaje kao što su `OnClick`, `OnMouseMove` i `OnKeyDown`, tako da su ovi događaji definisani u tipu `Control`, jer je roditelj svih kontrola i svi njegovi članovi ih nasleđuju. Detaljnije ćemo saznati o ovim događajima u sledećim pasusima, ali prvo moramo naučiti kako da obrađujemo događaje u sVB-u. Počnimo tako što ćemo na formu postaviti polje za tekst i dugme. Sada dvaput kliknite na dugme. Ovo će otvoriti uređivač koda i u njega ubaciti obrađivač događaja `OnClick`:



Konvencija je da se koristi ime `Control_Event` za potprogram koji obrađuje događaj, kao što je `Button1_OnClick` u našem primeru. Dakle, ako ste ručno dodali sledeći potprogram u datoteku koda, sVB će ga prepoznati kao obrađivač događaja za događaj `TextBox1.TextChanged`:

```
Sub TextBox1_OnTextChanged()
Me.Text = TextBox1.Text
EndSub
```

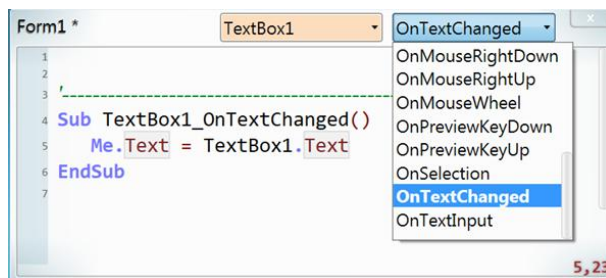
Sada, ako pritisnete F5 da biste pokrenuli program, svaki put kada promenite tekst napisan u polju za tekst, odmah ćete videti taj tekst na naslovnoj traci forme.

Ali, ne morate to da radite ručno, jer uređivač teksta se nudi da to uradi za vas. Na vrhu

datoteke koda videćete dve padajuće liste:

- Leva lista prikazuje ime forme i njene kontrole.
- Desna lista prikazuje događaje dostupne za kontrolu koja je trenutno izabrana na prvoj listi.

Dakle, možete jednostavno izabrati `textBox1` sa leve liste, a zatim izabrati događaj `OnTextChanged` sa desne liste, da bi se obrađivač dodao u kôd umesto vas. Imajte na umu da možete pritisnuti 't' na tastaturi da biste izabrali `OnTextChanged` sa desne liste, umesto da ga tražite. Možete lako obraditi bilo koji događaj za bilo koju kontrolu. Ali šta ako želite da napišete jedan pot-



program za obradu više od jednog događaja iste kontrole ili istog događaja za više kontrola? Možete razmisliti o pisanju koda u jednom potprogramu i pozivanju iz tih obrađivača događaja, što je sasvim validno, ali će to biti nepotrebno opširan kôd. Umesto toga, možemo reći sVB-u da samo jedan potprogram može da obrađuje više događaja, ali to se ne može uraditi alatima za uređivanje koda niti konvencijom imenovanja, i potrebno je da napišemo neki kôd da bismo dodelili obrađivač tim događajima. Upravo to je urađeno u projektu 'Mouse pos' u folderu sa primerima, gde smo sVB-u rekli da će se potprogram pod nazivom `OnMouseMove` koristiti za obradu događaja `OnMouseMove` forme i tri druge kontrole na njoj, koristeći ovaj kôd:

```
TextBox1.OnMouseMove = OnMouseMove
TextBox2.OnMouseMove = OnMouseMove
Label1.OnMouseMove = OnMouseMove
Me.OnMouseMove = OnMouseMove
Sub OnMouseMove()
    ' Pogledajte kod u aplikaciji
EndSub
```

Kada koristite jedan rukovalac za obradu događaja različitih kontrola, možete koristiti svojstvo `Event.SenderControl` da biste dobili kontrolu koja je pokrenula događaj. Generalno, postoje tri tipa koja će vam biti korisna prilikom obrade događaja kako bi vam pružila potrebne informacije o događaju: `Event`, `Mouse` i `Keyboard`.

**Napomena:** Možete registrovati samo jedan obrađivač za događaj, bilo podešavanjem pomoću koda ili korišćenjem konvencije imenovanja. Ako pokušate da dodate više od jednog obrađivača za isti događaj, koristiće se samo poslednji. Ali ovo važi samo u programskom domenu, tako da možete dodati još jedan obrađivač iz eksterne biblioteke. Na primer, kada pošaljete kontrolu metodi `Geometrics.AllowDrag`, ona će obraditi događaje `OnMouseMove` i `OnMouseDown` ove kontrole tako da je možete prevući na obrazac. Međutim, i dalje možete dodati obrađivač za događaj `OnMouseMove` i obrađivač za događaj `OnMouseDown` ove kontrole u vašem programu, što znači da će svaki od ovih događaja pozivati dva obrađivača, jedan kreiran od strane biblioteke `Geometrics`, a drugi kreiran od strane vašeg programa.

Tip kontrole ima sledeće članove:

### Control.Angle As Double

Dobija ili postavlja ugao rotacije kontrole. Ugao se meri u stepenima i možete koristiti negativne vrednosti i vrednosti veće od 360, ali kada pročitate ovo svojstvo, njegova vrednost će biti normalizovana da bude u opsegu od 0 do 259 stepeni.

Imajte na umu da ovaj ugao možete promeniti korišćenjem dugmeta za rotaciju na dizajneru obrazaca.

*Primeri:* Dodajte ComboBox i CheckBox na obrazac i napišite sledeći kôd u globalnom području forme:

```
ComboBox1.Angle = 400  
TextWindow.WriteLine(ComboBox1.Angle)      '40  
CheckBox1.Angle = -45  
TextWindow.WriteLine(CheckBox1.Angle)      '315
```

### Control.AnimateAngle(ugao, trajanje)

Animira ugao rotacije kontrole od trenutne vrednosti do datog novog ugla. To čini da se kontrola rotira oko svog centra tokom datog trajanja.

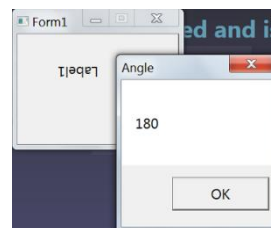
Za više detalja o tome kako animacija funkcioniše i kako je možete kontrolisati, pogledajte odeljak Animiranje kontrola.

*Parametri:*

- *ugao*: Novi ugao rotacije, koji će kontrola dostići nakon što se animacija završi.
- *trajanje*: Vreme animacije, u milisekundama.

*Primer:* Dodajte labelu na formu i napišite sledeći kôd u globalnom području forme:

```
Label1.AnimateAngle(180, 1000)  
Program.Delay(1000)  
Forms.ShowMessage(Label1.Angle, "Ugao")
```



Kada pokrenete program, oznaka će se rotirati u smeru kazaljke na satu jednu sekundu i zaustaviti se na uglu od 180 stepeni, a zatim će se u prozoru sa porukom prikazati poruka "180".

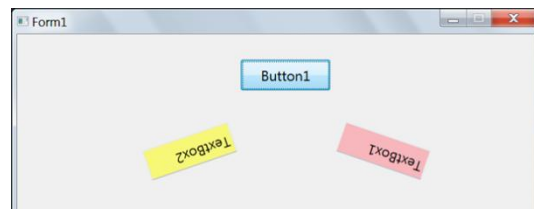
### Control.AnimateColor(novaBoja, trajanje)

Animira zadnju boju kontrole u novu boju. Za više detalja o tome kako animacija funkcioniše i kako je možete kontrolisati, pogledajte odeljak Animiranje kontrola.

*Parametri:*

- *novaBoja*: Nova boja za animaciju pozadinske boje kontrole.
- *trajanje*: Vreme animacije, u milisekundama.

*Primer:* Možete videti ovu metodu u akciji u projektu Animation2 u folderu samples, gde primenjujemo mnogo animacija na dva tekstualna polja da bismo ih rotirali i pomerili dok im se boje pozadine menjaju i njihova transparentnost se povećava.



### Control.AnimatePos(x, y, trajanje)

Animira kontrolu na novu poziciju. Za više detalja o tome kako animacija funkcioniše i kako je možete kontrolisati, pogledajte odeljak Animiranje kontrola.

*Parametri:*

- *x*: X koordinata nove pozicije. Svojstvo Control.Left će dostići ovu vrednost nakon završetka animacije.
- *y*: Y koordinata nove pozicije. Svojstvo Control.Top će dostići ovu vrednost nakon završetka animacije.
- *trajanje*: Vreme animacije, u milisekundama.

*Primer:* Možete videti ovu metodu u akciji u projektu Animation1 u folderu sa primerima, gde

animiramo ugao i poziciju dva tekstualna polja da bismo ih rotirali i pomerali.

### Control.AnimateSize(širina, visina, trajanje)

Animira kontrolu na novu veličinu. Za više detalja o tome kako animacija funkcioniše i kako je možete kontrolisati, pogledajte odeljak Animiranje kontrola.

*Parametri:*

- **širina**: Nova širina. Svojstvo Control.Width će dostići ovu vrednost nakon što se animacija završi.

- **visina**: Nova visina. Svojstvo Control.Height će dostići ovu vrednost nakon što se animacija završi.

- **trajanje**: Vreme animacije, u milisekundama.

*Primer:* Možete videti ovu metodu u akciji u projektu Animation3, da biste animirali veličinu elipse.

### Control.AnimateTransparency(prozirnost, trajanje)

Animira boju pozadine kontrole na novu transparentnost. Za više detalja o tome kako animacija funkcioniše i kako je možete kontrolisati, pogledajte odeljak Animiranje kontrola.

*Parametri:*

- **prozirnost**: Nova transparentnost za animiranje procenta transparentnosti boje pozadine. Koristite vrednost između 0 i 100.

- **trajanje**: Vreme animacije, u milisekundama.

*Primer:* Možete videti ovu metodu u akciji u projektu Animation2 u folderu sa primerima, gde primenjujemo mnoge animacije na dva tekstualna polja da bismo ih rotirali i pomerali dok se njihove boje pozadine menjaju i njihova transparentnost se povećava.

### Control.BackColor As Color

Dobija ili postavlja boju pozadine kontrole. Imajte na umu da će se lista automatskog dovršavanja pojaviti odmah nakon što otkucate = da biste ponudili imena dostupnih boja, a možete otkucati nekoliko znakova imena boje da biste filtrirali listu ili koristiti miš za pomeranje ili strelice gore i dole na tastaturi da biste se kretali kroz imena.

Kada pronađete željenu boju, dvaput kliknite na nju ili pritisnite Enter da biste je potvrdili. Ime će biti kvalifikovano prefiksom `Colors.` u uređivaču koda, kao što je pisanje `Colors.Transparent` kada izaberete `Transparent`. Takođe imajte na umu da možete otkazati listu automatskog dovršavanja pritiskom na taster Esc. Sada, ako napišete bilo koji znak, lista automatskog dovršavanja će se ponovo pojaviti, ali ovog puta neće nuditi nazive boja, već moguće stavke dovršavanja za trenutni kontekst, kako biste mogli da izaberete naziv promenljive ili funkcije. Ako želite ponovo da vidite nazive fontova, obrišite sve znakove posle = i pritisnite Ctrl+Space.

*Primer:* Dodajte ListBox na obrazac i napišite ovaj kôd u globalnoj oblasti:

```
ListBox1.AddItem({1, 2, 3})
ListBox1.BackColor = Colors.Yellow
```

Label1.BackColor = tr



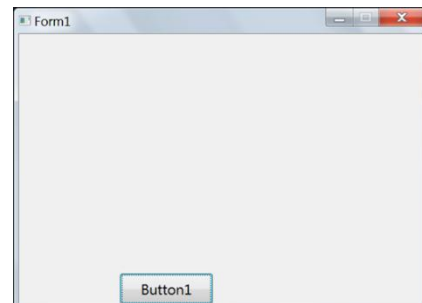
## Control.Bottom As Double

Dobija ili postavlja y-poziciju donje ivice kontrole u odnosu na gornju ivicu njene nadređene kontrole (forme). Ako koristite negativnu vrednost, kontrola će biti van forme.

*Primer:* Dodajte dugme na formu, kliknite dvaput na njega i napišite ovaj kôd u njegovom OnClick handler-u:

```
Button1.Bottom = Me.Height
```

Kada pokrenete program i kliknete na dugme, vertikalni položaj dugmeta će se promeniti tako da njegova donja ivica dodiruje donju ivicu forme.



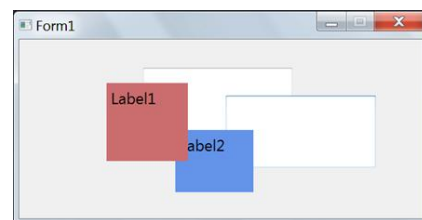
## Control.BringToFront( )

Dovodi trenutnu kontrolu ispred svih ostalih kontrola. Ako je kontrola forma, biće aktivirana.

*Primer:* Dodajte 2 tekstualna polja i 2 labele na formu i dizajnirajte je ovako:

Pređite na uređivač koda i napišite ovaj kôd:

```
TextBox1.OnMouseLeftDown = OnMouseLeftDown
TextBox2.OnMouseLeftDown = OnMouseLeftDown
Label1.OnMouseLeftDown = OnMouseLeftDown
Label2.OnMouseLeftDown = OnMouseLeftDown
Sub OnMouseLeftDown()
    control1 = Event.SenderControl
    If Keyboard.CtrlPressed Then
        control1.SendToBack()
    Else
        control1.BringToFront()
    EndIf
EndSub
```



Imajte na umu da smo koristili potprogram OnMouseLeftDown za obradu događaja OnMouseLeftDown za 4 kontrole, umesto da ponavljamo isti kôd 4 puta u 4 različita obrađivača sa promenjenim samo imenom kontrole.

U potprogramu OnMouseLeftDown, koristili smo svojstvo Event.SenderControl da bismo dobili kontrolu koja je pokrenula događaj, tako da je možemo prebaciti u prvi plan ili u pozadinu u skladu sa stanjem tastera Ctrl.

Sada pritisnite F5 da biste pokrenuli projekat i pokušajte ovo:

- Kliknite na bilo koju kontrolu da biste je prebacili u prvi plan.
- Pritisnite taster Ctrl i izaberite bilo koju kontrolu da biste je poslali u pozadinu svih ostalih kontrola.

## Control.CaptureMouse( )

Čini da trenutna kontrola poseduje miš i njegove događaje, dok ne pozovete funkciju ReleaseMouse.

## Control.ChooseBackColor( ) As Color

Prikazuje dijalog za boje kako bi korisnik mogao da promeni boju pozadine trenutne kontrole.

*Vraća:* Boja koju korisnik izabere ili "" ako je otkazao operaciju.



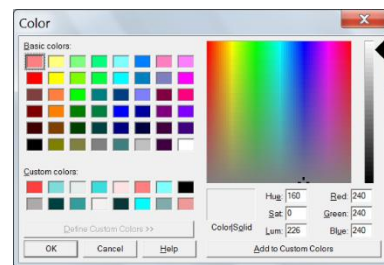
U većini slučajeva, ignorisaćete ovu povratnu vrednost, osim ako ne želite da promenite boju mnogih kontrola.

*Primer:* Dodajte TextBox i Button na formu. Dvaput kliknite na dugme i upišite ovaj kôd u njegovu funkciju za obradu događaja OnClick:

```
TextBox1.ChooseBackColor()
```

Pritisnite F5 da biste pokrenuli program i kliknite na dugme. Ovo će prikazati dijalog sa bojama. Izaberite boju i kliknite na OK. Boja koju izaberete biće primenjena na boju pozadine tekstualnog polja.

Kliknite na dugme da biste ponovo prikazali dijalog sa bojama i imajte na umu da je boja pozadine tekstualnog polja prvobitno izabrana. Izaberite drugu boju, ali ovaj put kliknite na dugme za otkazivanje. Boja pozadine tekstualnog polja se ovog puta neće promeniti.



### Control.ChooseFont( ) As Array

Prikazuje dijalog fonta kako bi korisnik mogao da promeni svojstva fonta, uključujući boju prednjeg plana trenutne kontrole.

*Vraća:* Ako korisnik zatvori dijalog fonta klikom na dugme OK, ovaj metod će vratiti niz koji sadrži svojstva fonta pod ključevima Name – *Ime*, Size – *Veličina*, Bold – *Podebljano*, Italic – *Kurziv*, Underline – *Podvučeno* i Color – *Boja*.

U većini slučajeva, ignorisaćete ovu povratnu vrednost, osim ako niste odlučili da promenite svojstva fonta mnogih kontrola, pa možete postaviti ovu povratnu vrednost na svojstvo Font svake od ovih kontrola ili možete pročitati pojedinačne ključeve iz vraćenih nizova da biste postavili vrednosti pojedinačnih svojstava nekih kontrola ako želite.

Ako korisnik otkáže dijalog, ova metoda će vratiti prazan string "".

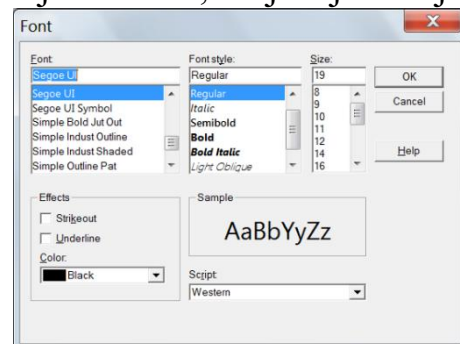
*Primer:* Dodajte TextBox i Button na obrazac. Dvaput kliknite na dugme i upišite ovaj kôd u njegov program za obradu događaja OnClick:

```
TextBox1.ChooseFont()
```

Pritisnite F5 da biste pokrenuli program i kliknite na dugme. Ovo će prikazati dijalog fontova. Izaberite naziv fonta i stilove, a zatim kliknite na OK. Svojstva fonta koja izaberete biće primenjena na svojstva FontName, FontBold, FontItalic, Underlined i ForeColor polja za tekst.

Kliknite na dugme da biste ponovo prikazali dijalog fontova i imajte na umu da su vrednosti svojstava fonta polja za tekst prvobitno izabrane.

Izaberite druge vrednosti, ali ovaj put kliknite na dugme za otkazivanje. Font polja za tekst se ovog puta neće promeniti.



### Control.ChooseForeColor( ) As Color

Prikazuje dijalog za boje kako bi korisnik mogao da promeni boju prednjeg plana trenutne kontrole.

*Vraća:* Boja koju je korisnik izabrao ili "" ako je otkazao operaciju.

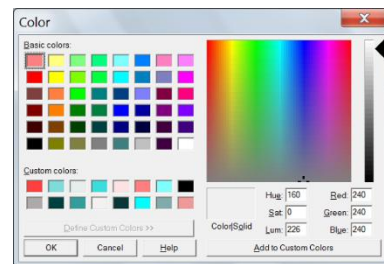
U većini slučajeva, ignorisaćete ovu povratnu vrednost, osim ako ne želite da promenite boje mnogih kontrola.

*Primer:* Dodajte TextBox i Button na obrazac. Dvaput kliknite na dugme i napišite ovaj kôd u njegovom OnClick obrađivaču događaja:

```
TextBox1.ChooseForeColor()
```

Pritisnite F5 da biste pokrenuli program i kliknite na dugme. Ovo će prikazati dijalog za boje. Izaberite boju i kliknite na OK (U redu). Boja koju izaberete biće primenjena na boju prednjeg plana tekstualnog polja.

Kliknite na dugme da biste ponovo prikazali dijalog za boje i imajte na umu da je boja prednjeg plana tekstualnog polja prvobitno izabrana. Izaberite drugu boju, ali ovaj put kliknite na dugme za otkazivanje. Boja prednjeg plana tekstualnog polja se neće promeniti.



## Control.Enabled As Boolean

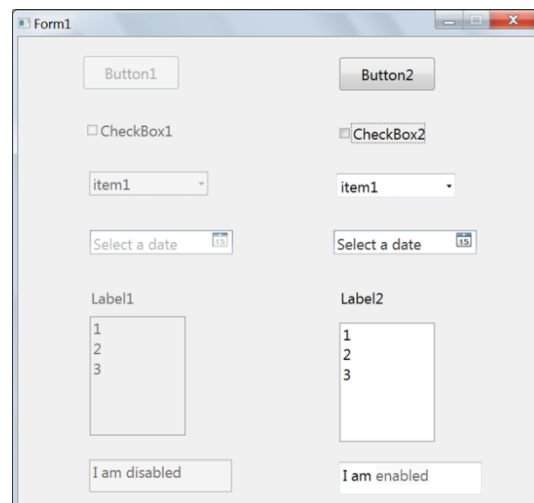
Dobija ili postavlja vrednost da li je kontrola omogućena ili onemogućena.

Kada je vrednost True, korisnik može da interaguje sa kontrolom. Kada je vrednost False, kontrola je zatamnjena da bi se naznačilo da je onemogućena i korisnik ne može da interaguje sa njom.

*Primer:* Ova slika vam prikazuje kako kontrole izgledaju kada su onemogućene (levo) i kada su omogućene (desno):

Da biste ovo isprobali, dodajte nekoliko kontrola koje vidite na slici na obrascu i napišite ovaj kôd u globalnom području obrasca:

```
ComboBox1.Text = "item1"  
ComboBox2.Text = "item1"  
ListBox1.AddItem({1, 2, 3})  
ListBox2.AddItem({1, 2, 3})  
TextBox1.Text = "Onemogućen sam"  
TextBox2.Text = "Omogućen sam"  
Button1.Enabled = False  
CheckBox1.Enabled = False  
ComboBox1.Enabled = False  
DatePicker1.Enabled = False  
Label1.Enabled = False  
ListBox1.Enabled = False  
TextBox1.Enabled = False
```



## Control.Error As String

Dobija ili postavlja poruku o grešci za ovu kontrolu.

Kada podesite poruku o grešci, kontrola će prikazati crveni okvir, a poruka o grešci će se prikazati kao savet alatke kada miš pređe preko kontrole.

Da biste resetovali grešku, samo postavite ovo svojstvo na prazan string.

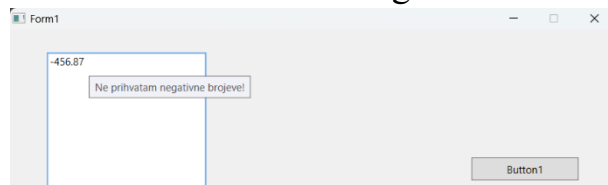
Svojstvo Error – *Greška* je namenjeno za validaciju kontrola unosa pre nego što se podaci korisničkog unosa pošalju vašem programu radi obrade. Možete validirati sve kontrole pomoću obrađivača događaja klika na dugme, ali ako obrazac sadrži mnogo kontrola, sastavljanje celog koda za validaciju zajedno može nepotrebno zakomplikovati kôd, pa se u sVB preporučuje dodavanje koda za validaciju u svaki obrađivač događaja OnLostFocus kontrole, kako bi korisnik mogao biti obavešten da je uneo nevažeću vrednost čim napusti kontrolu.

Za više informacija o validaciji kontrola, pogledajte metod Form.Validate.



*Primer:* Svojstvo Error možete videti u akciji u projektu Validacija u fascikli primeri. Na primer, koristi se u obradi događaja OnLostFocus elementa TextBox2 da bi se izdala greška ako korisnik unese bilo šta osim pozitivnih brojeva:

```
Sub TextBox2_OnLostFocus()  
n = TextBox2.Text  
If Text.IsNumeric(n) = False Then  
    TextBox2.Error = "Prihvatam samo cifre!"  
ElseIf n < 0 Then  
    TextBox2.Error = "Ne prihvatam negativne brojeve!"  
Else  
    TextBox2.Error = ""  
EndIf  
EndSub
```



Imajte na umu da morate postaviti svojstvo Error na string "" kada je unos validan, da biste resetovali kontrolu na njen normalan stil nakon što je korisnik popravio ulazne podatke. Zbog toga smo koristili blok Else u gornjem kodu.

### Control.FitContentHeight( )

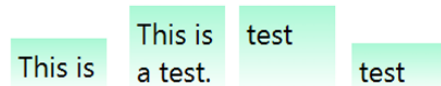
Menja visinu kontrole kako bi se uklopila u visinu njenog sadržaja. Ovo može biti korisno kada podesite WordWrap = True u kontrolama label, textbox i button.

Ovo je jednokratna promena i neće automatski podesiti veličinu visine kontrole. Ako želite da se visina automatski podesi, podesite svojstvo Height – *Visina* na -1.

*Primer:* Ovo svojstvo možete videti u akciji u projektu „Fit Size“ – *Prilagodi veličini* u folderu sa primerima. Dugme „Fit Height“ – *Prilagodi visini* ima ovaj jedan red u svom obrađivaču događaja OnClick:

```
Label1.FitContentHeight()
```

Pokrenite program, a zatim kliknite na dugme „Long Text“ – *Dugi tekst* da biste tekst učinili većim od širine labele. Nećete videti ceo tekst, jer labela neće promeniti svoju veličinu da bi se uklopila u tekst. Sada kliknite na dugme „Fit Height“ da biste labeli promenili svoju visinu da bi se uklopila u tekst. Ali ovo je jednokratna promena, tako da, ako kliknete na dugme „Short Text“ – *Kratak tekst*, veličina labele se neće promeniti iako se tekst smanjio. Možete smanjiti visinu labele da bi se uklopila u ovaj kratki tekst ponovnim klikom na dugme „Fit Height“. Četiri rezultata su prikazana na slici desno:



### Control.FitContentSize( )

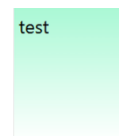
Menja širinu i visinu kontrole kako bi se uklopila u veličinu njenog sadržaja.

Ovo je jednokratna promena i neće automatski podesiti veličinu širine i visine kontrole. Ako želite da ih automatski podesite, postavite svojstva Width i Height na -1.

*Primer:* Ovo svojstvo možete videti u akciji u projektu „Fit size“ u folderu samples. Dugme 'Fit Size' ima ovu jednu liniju u svom OnClick obradi događaja:

```
Label1.FitContentSize()
```

Pokrenite program, a zatim kliknite na ovu dugmad i vidite šta se dešava: 'Short Text', 'Fit Size', 'Long Text', 'Fit Height', 'Fit Width', 'Short Text'. Sada labela izgleda (slika desno gore). Na kraju, kliknite na dugme 'Fit Size' da bi oznaka izgledala ovako:



## Control.FitContentWidth( )

Menja širinu kontrole kako bi se uklopila u širinu njenog sadržaja. Ovo je jednokratna promena i neće automatski podesiti veličinu širine kontrole. Ako želite da se veličina širine automatski podesi, podesite svojstvo Width na -1.

*Primer:* Ovo svojstvo možete videti u akciji u projektu „Fit size“ u folderu samples. Dugme `Fit Width` ima ovu jednu liniju u svom OnClick rukovaocu događaja:

```
Label1.FitContentWidth()
```

Pokrenite program, a zatim kliknite na ovu dugmad i vidite šta se dešava: `Long Text`, `Fit Height`, `Fit Width`.

This is a test.

Sada oznaka izgleda ovako:

## Control.Focus( )

Pomera fokus na kontrolu, tako da ona postaje aktivna kontrola koja prima tastere sa tastature.

*Primer:* Ovu metodu možete videti u akciji u projektu `Simple Calculator` u folderu samples. Koristi se u OnClick rukovaocu dugmeta za deljenje da bi se postavio fokus txtNum2 kada korisnik unese 0 u njega, jer deljenje sa nulom nije moguće:

```
Sub BtnDivide_OnClick  
Ako txtNum2.Text = 0 Then  
result = "Ne mogu da delim sa nulom."  
txtNum2.SelectAll()  
txtNum2.Focus()  
Else  
result = "Result = " + (  
txtNum1.Text / txtNum2.Text)  
EndIf  
lblResult.Text = result  
EndSub
```

## Control.Font As Array

Dobija ili postavlja niz koji sadrži svojstva fonta pod ključevima: Ime, Veličina, Podebljano, Kurziv, Podvučeno i Boja, tako da ih možete koristiti kao indeksere nizova ili dinamička svojstva. Ne morate da podešavate sve ove ključeve, jer će nedostajući ključevi zadržati odgovarajuća svojstva fonta nepromenjenim. Međutim, malo je verovatno da ćete ove ključeve podesiti ručno, jer je svojstvo fonta namenjeno za upotrebu u dva slučaja:

1. Kopiranje svojstava fonta sa jedne kontrole na drugu, korišćenjem samo ove jedne linije koda:

```
Label1.Font = TextBox1.Font
```

2. Postavljanje ovog svojstva na izabrani font koji vraća metoda Desktop.ShowFontDialog. U suprotnom, trebalo bi da koristite pojedinačna svojstva fonta, koja su navedena u nastavku.

## Control.FontBold As Boolean

Dobija ili postavlja da li je font koji se koristi za prikaz teksta kontrole podebljan (Bold).

*Primer:* Dodajte TextBox na formu i napišite ovu liniju koda u globalno područje forme:

```
TextBox1.FontBold = True
```

Pokrenite program i napišite neke znakove u TextBox. Pojaviće se podebljanim fontom.

Hello

## Control.FontItalic As Boolean

Dobija ili postavlja da li je font koji se koristi za prikaz teksta kontrole kurziv (Italic).

*Primer:* Dodajte TextBox na obrazac i napišite ovu liniju koda u globalnom području forme:

```
TextBox1.FontBoldItalic = True
```

Pokrenite program i napišite nekoliko znakova u TextBox.

Oni će se pojaviti u kurzivnom stilu.

Hello

## Control.FontName As String

Dobija ili postavlja ime fonta koje se koristi za prikaz teksta kontrole. Lista automatskog dovršavanja će se pojaviti odmah nakon što otkucate = da biste ponudili imena dostupnih sistemskih fontova, a možete otkucati nekoliko znakova imena fonta da biste filtrirali listu ili koristiti miš za pomeranje ili strelice gore i dole na tastaturi da biste se kretali kroz imena.

Kada pronađete željeni font, dvaput kliknite na njega ili pritisnite Enter da biste ga potvrdili. Ime će biti pod navodnicima u uređivaču koda:

```
TextBox1.FontName = "Old Antic Bold"
```

Imajte na umu da možete otkazati listu automatskog dovršavanja pritiskom na taster Esc. Sada, ako napišete bilo koji znak, lista automatskog dovršavanja će se ponovo pojaviti, ali ovaj put neće nuditi imena fontova, već moguće stavke za dovršavanje za trenutni kontekst, kako biste mogli da izaberete ime promenljive ili funkcije. Ako želite ponovo da vidite imena fontova, obrišite sve znakove posle = i pritisnite Ctrl+Space. Ili možete da otkucate " or "" i kada napišete znak posle početnog navodnika, imena fontova će se pojaviti na listi za dovršavanje, a kada unesete ime, završni navodnik će biti dodat ako nedostaje.



## Control.FontSize As Double

Dobija ili postavlja veličinu fonta koja se koristi za prikaz teksta kontrole.

Ova veličina se meri u poenima, kao što ste navikli u Windows aplikacijama kao što je MS Word. Možete koristiti celobrojne vrednosti između 1 i 100.

Imajte na umu da veličina fonta zavisi i od tipa fonta. Na primer, obe sledeće reči imaju veličinu fonta od 36 poena, ali imaju različite tipove fontova („Times New Roman“ i „Segoe UI“), tako da ne izgledaju jednako:

Sample  
Sample

## Control.ForeColor As Color

Boja prednjeg plana koja se koristi za crtanje teksta kontrole.

Koristite vrednosti iz objekta Color, kao što je Color.Red.

Imajte na umu da će se lista automatskog dovršavanja pojaviti odmah nakon što otkucate = da biste ponudili imena dostupnih boja, a možete otkucati nekoliko znakova imena boje da biste filtrirali listu ili koristiti miš za pomeranje ili strelice gore i dole na tastaturi da biste se kretali kroz imena.

Kada pronađete željenu boju, dvaput kliknite na nju ili pritisnite Enter da biste je potvrdili.



Ime će biti kvalifikovano prefiksom `Colors.` u uređivaču koda, kao što je pisanje `Colors.Blue` kada izaberete `Blue`.

Takođe imajte na umu da možete otkazati listu automatskog dovršavanja pritiskom na taster Esc. Sada, ako napišete bilo koji znak, lista automatskog dovršavanja će se ponovo pojaviti, ali ovog puta neće nuditi nazive boja, već moguće stavke dovršavanja za trenutni kontekst, kako biste mogli da izaberete naziv promenljive ili funkcije. Ako želite ponovo da vidite nazive fontova, obrišite sve znakove posle = i pritisnite Ctrl+Space.

*Primer:* Dodajte dugme na formu i napišite ovaj kôd u njenu globalnu oblast:

```
Button1.ForeColor = Colors.Blue
```

Kada pokrenete program, tekst dugmeta će imati plavu boju.

## Control.Height As Double

Dobija ili postavlja visinu kontrole.

Ako želite da koristite automatsku visinu da bi se uklopila u visinu sadržaja kontrole, postavite ovo svojstvo na -1.

Takođe možete ograničiti automatsku visinu da ne prelazi određenu visinu podešavanjem svojstva MaxHeight.

## Control.Left As Double

Dobija ili postavlja x-poziciju leve ivice kontrole u odnosu na levu ivicu njene nadređene kontrole (forme). Ako koristite negativnu vrednost, leva ivica kontrole će biti van forme, a ako leva postane negativnija, cela kontrola se možda neće pojaviti na formi.

*Primer:* Dodajte dva dugmeta na formu i napišite ovaj kôd:

```
Button2.Left = -Button1.Width  
Sub Button1_OnClick()  
Button2.Left = Button2.Left + Button1.Width / 2  
EndSub
```



Kada pokrenete program, dugme2 se neće pojaviti na formi, jer je njegova leva = - njegova širina. Kada kliknete na dugme1, levo dugme2 će se povećati za polovinu svoje širine, tako da će se njegova polovina pojaviti na formi. Kada ponovo kliknete na dugme1, dugme2 će se u potpunosti pojaviti na formi. Možete nastaviti da klikćete na dugme button1 da biste povećali horizontalni položaj dugmeta button2, dok ne pređe desnu ivicu forme i nestane.

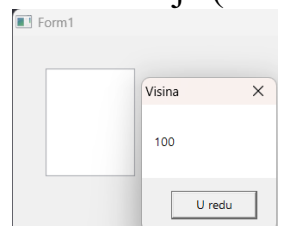
## Control.MaxHeight As Double

Dobija ili postavlja maksimalnu visinu kontrole. Korisno je posebno kada podesite visinu kontrole na automatsku (Height = -1), da biste primorali automatsku visinu da ne prelazi maksimalnu dužinu.

Imajte na umu da će postavljanje ovog svojstva na 0 ili negativnu vrednost resetovati je (bez maksimalne visine).

*Primer:* Dodajte TextBox na formu i napišite ovaj kôd u njenu globalnu oblast:

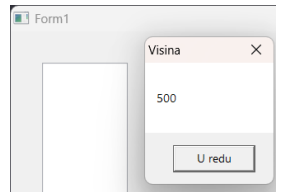
```
TextBox1.MaxHeight = 100  
TextBox1.Height = 500  
Me.ShowMessage(TextBox1.Height, "Visina")  
TextBox1.MaxHeight = 0
```



```
Me.ShowMessage(TextBox1.Height, "Visina")
```

Kada pokrenete program, okvir za poruku će vam reći da je visina tekstualnog polja 100, što je maksimalna dozvoljena visina.

To znači da ne možete nametnuti visinu kontrole veću od njene maksimalne visine. Nakon što zatvorite okvir za poruku, maksimalna visina će biti resetovana, što će omogućiti visini tekstualnog okvira da pređe 100, tako da će drugi okvir za poruku prikazivati 500! To znači da je vrednost koju ste postavili za svojstvo Height sačuvana, ali će čitanje svojstva Height uvek vraćati stvarnu visinu kontrole.



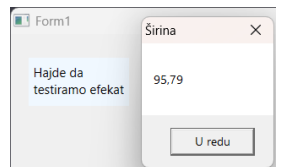
### Control.MaxWidth As Double

Maksimalna širina kontrole. Posebno je korisno kada podesite širinu kontrole na automatsku (Širina = -1), da biste naterali automatsku širinu da ne prelazi maksimalnu dužinu.

Imajte na umu da će postavljanje ovog svojstva na 0 ili negativnu vrednost resetovati njegovu vrednost (bez maksimalne širine).

*Primer:* Dodajte labelu na formu i napišite ovaj kôd u globalnom području forme:

```
Label1.MaxWidth = 100  
Label1.BackColor = Colors.AliceBlue  
Label1.Width = -1  
Label1.Text = "Hajde da testiramo efekat MaxWidth"  
Me.ShowMessage(Label1.Width, "Širina")
```



Kada pokrenete program, prozor sa porukom će vam reći da je širina labele 100 ili malo manja: Ali zašto širina može biti < 100? Zar širina labele ne može biti tačno 100?

Odgovor možete dobiti ako dodate ovu liniju koda na početak gornjeg koda:

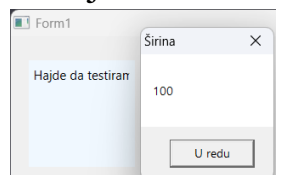
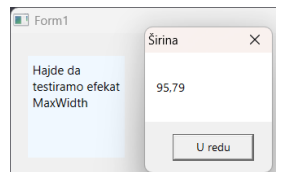
```
Label1.Height = 100
```

Kada pokrenete program, moći ćete da vidite da je ostatak testa prelomljen preko sledećih redova. Svaki prelomljeni red počinje celom rečju, tako da se svaki fragmentirani red završava razmakom, a širina oznake se podešava tako da odgovara najdužem od njih.

Dakle, ako želite da imate punu maksimalnu širinu, isključite prelom reči dodavanjem ove linije koda na početak koda:

```
Label1.WordWrap = False
```

Kada pokrenete program, dobićete ono što očekujete (uz cenu odsecanja dodatnog teksta iz oblasti labele):



### Control.MouseX As Double

Dobija x-poziciju miša u odnosu na kontrolu. Kada se miš nalazi iznad kontrole, ova vrednost se nalazi između 0 i širine kontrole. Negativna vrednost znači da je pokazivač miša van leve ivice kontrole.

Imajte na umu da možete koristiti svojstvo Mouse.X da biste dobili x-poziciju miša u odnosu na ekran.

*Primer:* Možete videti ovo svojstvo u akciji u projektu 'Mouse pos' u folderu sa primerima. Koristi se u obrađivaču događaja OnMouseMove svake od 3 kontrole i forme da bi se prikazao položaj miša u odnosu na svaku od njih. Ovde ćemo prikazati samo za TextBox1:

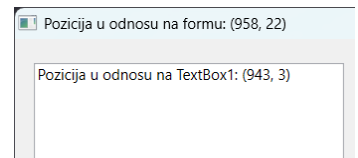
```
TextBox1.Text = Text.Format(  
"Pozicija u odnosu na TextBox2: ([1], [2])",
```



```

{TextBox1.MouseX, TextBox1.MouseY}
)
Form1.Text = Text.Format(
"Pozicija u odnosu na formu: ([1], [2])",
{Me.MouseX, Me.MouseY}
)

```



## Control.MouseY As Double

Dobija y-poziciju miša u odnosu na kontrolu. Kada je miš iznad kontrole, ova vrednost se nalazi između 0 i visine kontrole. Negativna vrednost znači da je pokazivač miša iznad gornje ivice kontrole.

Imajte na umu da možete koristiti svojstvo `Mouse.Y` da biste dobili x-poziciju miša u odnosu na ekran.

*Primer:* Možete videti ovo svojstvo u akciji u projektu 'Mouse pos' u folderu sa primerima. Koristi se u obrađivaču događaja `OnMouseMove` svake od 3 kontrole i forme da bi se prikazao položaj miša u odnosu na svaku od njih. Ovde ćemo prikazati samo za `Textbox1` (isti primer kao prethodni):

```

TextBox1.Text = Text.Format(
"Pozicija u odnosu na TextBox1: ([1], [2])",
{TextBox1.MouseX, TextBox1.MouseY}
)
Form1.Text = Text.Format(
"Pozicija u odnosu na formu: ([1], [2])",
{Me.MouseX, Me.MouseY}
)

```

## Control.Name As String

Dobija ime kontrole, što je jedinstveni identifikator koji koristimo u kodu za referencu na kontrolu, kao što su `TextBox1` i `BtnOk`.

Ne možete promeniti ime kontrole tokom izvršavanja. Koristite dizajner da biste izabrali kontrolu, a zatim koristite gornje polje za tekst 'Name' da biste promenili ime:

Imajte na umu da je promenljiva kontrole zapravo string, koji se sastoji od imena forme i imena kontrole razdvojenih tačkom kao što je 'form1.textbox1', dok svojstvo `Name` kontrole sadrži samo njeno ime bez imena forme.

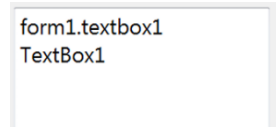
*Primer:* Dodajte `TextBox` na formu i napišite ovaj kod u globalnom području forme:

```

TextBox1.AppendLine(TextBox1)
TextBox1.AppendLine(TextBox1.Name)

```

Kada pokrenete program, polje za tekst će se pojaviti kao na ovoj slici:



## Control.OnClick

Ovaj događaj se pokreće kada korisnik pritisne levi taster miša na kontroli, a zatim ga otpusti.

Ovo je podrazumevani događaj za kontrole `Button` i `Label`, tako da možete samo dvaput kliknuti na njih u dizajneru forme da biste dodali rukovaoca za ovaj događaj, kao što je:

```

Sub Button1_OnClick()
EndSub

```

Za više informacija, pročitajte o rukovanju događajima kontrole.

## Control.OnDoubleClick

Ovaj događaj se pokreće kada korisnik dvaput klikne na kontrolu. Za više informacija, pročitajte o rukovanju događajima kontrole.

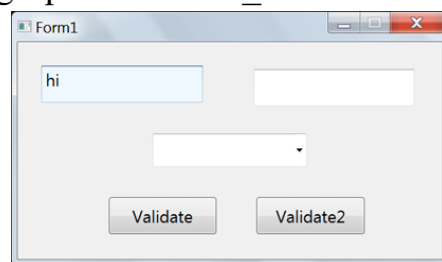
## Control.OnGotFocus

Ovaj događaj se pokreće kada kontrola dobije fokus.

Za više informacija, pročitajte o rukovanju događajima kontrole.

*Primer:* Možete videti ovaj događaj u akciji u projektu `Validation` u folderu sa primerima. Koristi se da bi se fokusiranoj kontroli dala drugačija pozadina. I TextBox2 i ComboBox1 će koristiti obrađivač događaja OnGotFocus za TextBox1, tako da će podgrupa TextBox1\_OnGotFocus obrađivati 3 kontrole:

```
TextBox2.OnGotFocus = TextBox1_OnGotFocus
ComboBox1.OnGotFocus = TextBox1_OnGotFocus
'-----
Sub TextBox1_OnGotFocus()
    senderControl = Event.SenderControl
    senderControl.BackColor = Colors.AliceBlue
EndSub
```



Imajte na umu da moramo vratiti belu pozadinsku boju kontroli kada je napustimo, tako da moramo koristiti događaj OnLostFocus za svaku kontrolu da bismo to uradili, ali u ovom trenutku nećemo koristiti jedan obrađivač za 3 kontrole, jer projekat Validation koristi OnLostFocus za svaku kontrolu da bi validirao njen sadržaj, a logika validacije je različita za svaku kontrolu, tako da ako zaključate taj projekat, možete videti da Rukovalac OnLostFocus za svaku kontrolu proverava pravila validacije, a zatim resetuje boju pozadine. Na primer:

```
Sub TextBox1_OnLostFocus()
    If TextBox1.Text = "" Then
        TextBox1.Error = "Ne drži me praznim!"
    Else
        TextBox1.Error = ""
    EndIf
    TextBox1.BackColor = Colors.White
EndSub
```

## Control.OnKeyDown

Ovaj događaj se pokreće kada korisnik pritisne taster na tastaturi.

Koristite svojstva tastature da biste dobili informacije o tasteru koji je pokrenuo događaj i stanju tastera Ctrl, Shift, Alt i drugih specijalnih tastera.

Imajte na umu da pritiskanje F10 ima isti efekat kao pritiskanje Alt, pa ako želite da koristite F10 u drugom zadatku u vašem programu, ne zaboravite da podesite svojstvo Event.Handled na True da biste sprečili Windows da dalje obrađuje taster F10 nakon što izvršite zadatak.

Za više informacija, pročitajte o rukovanju događajima kontrole.

*Primer:* Možete videti ovaj događaj u akciji u projektu `Numeric TextBox` u folderu sa primerima. Koristi se da spreči korisnika da ukuca bilo šta osim cifara u tekstualno polje. Da bismo to uradili, koristimo Keyboard.LastKey svojstvo da bi se dobio taster koji je pokrenuo događaj, i uverite se da je to jedan od cifarskih tastera (iznad slva tastature!), koji su predstavljeni svojstvima `Keys` od `Key.D0` do `Key.D9`. Ako je pritisnuti taster van ovog opsega, to moramo sprečiti postavljanjem `Event.Handled` na `True` da bismo rekli tekstualnom polju da više ništa ne radi, što

znači da neće pisati pritisnuti taster. Ovo je kôd `OnKeyDown` obrađivača:

```
Sub TextBox1_OnKeyDown()  
lastKey = Keyboard.LastKey  
If lastKey < Keys.D0 Or lastKey > Keys.D9 Then  
Event.Handled = True  
EndIf  
EndSub
```

### Control.OnKeyUp

Ovaj događaj se pokreće kada korisnik otpusti pritisnut taster na tastaturi. Koristite svojstva Keyboard – *tastature* da biste dobili informacije o tasteru koji je pokrenuo događaj i stanju tastera Ctrl, Shift, Alt i drugih specijalnih tastera. Za više informacija, pročitajte o rukovanju događajima kontrole. Imajte na umu da pritisak na F10 ima isti efekat kao i pritisak na Alt, pa ako želite da koristite F10 u drugom zadatku u vašem programu, ne zaboravite da podesite svojstvo Event.Handled na True da biste sprečili Windows da dalje obrađuje taster F10 nakon što izvršite zadatak.

### Control.OnLostFocus

Ovaj događaj se pokreće kada kontrola izgubi fokus. Koristite događaj da biste potvrdili korisnički unos kako biste bili sigurni da je uneo važeće podatke. Za više informacija, pogledajte kako da koristite svojstvo greške u ovom rukovaocu događajima za validaciju kontrole. Za više informacija, pročitajte o rukovanju događajima kontrole.

### Control.OnMouseEnter

Ovaj događaj se pokreće kada pokazivač miša uđe u oblast kontrole. Koristite svojstva miša da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

### Control.OnMouseLeave

Ovaj događaj se pokreće kada pokazivač miša napusti područje kontrole.

Koristite svojstva Mouse da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

### Control.OnMouseDown

Ovaj događaj se pokreće kada korisnik pritisne levi taster miša. Koristite svojstva Mouse da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

### Control.OnMouseLeftUp

Ovaj događaj se pokreće kada korisnik otpusti levi taster miša nakon što je pritisnut.

Koristite svojstva Mouse da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

## Control.OnMouseMove

Ovaj događaj se pokreće više puta dok se pokazivač miša kreće preko kontrole. Koristite svojstva Mouse da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

*Primer:* Možete videti ovaj događaj u akciji u projektu 'FormShape' u folderu sa primerima. Koristi se da omogući korisniku da prevuče formu klikom levog tastera miša na bilo koju tačku i pomeri miš na novu poziciju, a zatim otpusti dugme miša. Da bismo to uradili, moramo sačuvati poslednju tačku da bismo pomerili formu za razliku između pozicije miša i te tačke. Ovo je kôd:

```
LastX = 0
LastY = 0
Sub Form1_OnMouseMove()
If Event.IsLeftButtonDown Then
Me.Left = Me.Left + Mouse.X - LastX
Me.Top = Me.Top + Mouse.Y - LastY
EndIf
LastX = Mouse.X
LastY = Mouse.Y
EndSub
```

## Control.OnMouseDown

Ovaj događaj se pokreće kada korisnik pritisne desni taster miša.

Koristite svojstva Mouse – *miša* da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

## Control.OnMouseUp

Ovaj događaj se pokreće kada korisnik otpusti desni taster miša nakon što je pritisnut.

Koristite svojstva Mouse da biste dobili informacije o položaju miša i stanju njegovih dugmadi.

Za više informacija, pročitajte o rukovanju događajima kontrole.

## Control.OnMouseWheel

Ovaj događaj se pokreće više puta dok korisnik pomera točak miša.

Koristite svojstvo Event.LastMouseWheelDirection da biste saznali da li se točak pomera gore ili dole.

Za više informacija, pročitajte o rukovanju događajima kontrole.

*Primer:* Dodajte labelu na formu, promenite joj boju pozadine i prednjeg dela da izgledaju kao što vidite na sledećoj slici i postavite njen tekst na 0.

Pređite na uređivač koda i dodajte ovaj kôd da biste povećali ili smanjili vrednost labele kada korisnik pomera točak miša gore ili dole, respektivno:

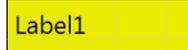
```
N = 0
Sub Label1_OnMouseWheel()
N = N + Event.LastMouseWheelDirection
Label1.Text = N
EndSub
```



## Control.Padding As Double

Dobija ili postavlja rastojanje kontrole, što je unutrašnji prostor između ivica kontrole i njenog sadržaja.

*Primer:* Dodajte labelu na formu, promenite joj boju pozadine i okvir da izgledaju kao što vidite na sledećoj slici.



Podrazumevano, postoji rastojanje od 5 tačaka između okvira labele i njenog teksta. Možete povećati ovo popunjavanje na 25 koristeći ovaj kôd:

```
Label1.Padding = 25  
Label1.FitContentSize()
```



Imajte na umu da promena popunjavanja može prouzrokovati nestanak dela teksta ako su visina, širina ili oznaka mali, pa smo pozvali metod FitContentSize da bismo prilagodili veličinu labele kako bi se tekst uklopio sa novim popunjavanjem.

Imajte na umu da popunjavanje može smetati kada dodate sliku na labelu, pa ga možete ukloniti postavljanjem ovog svojstva na 0:

```
Label1.Padding = 0
```

## Control.ReleaseMouse( )

Otpušta miš ako ga je trenutna kontrola snimila pozivanjem metode CaptureMouse.

## Control.RemoveEventHandler( )

Uklanja obrađivač datog događaja trenutne kontrole. Utiče samo na obrađivače događaja koje je dodao vaš program i neće ukloniti nijedan obrađivač koji je dodat od strane eksterne sVB biblioteke. Ako ste napisali biblioteku koja dodaje rukovaoce događajima, možete obezbediti funkciju koja uklanja ove rukovaoce, tako da korisnik vaše biblioteke može da je pozove. Pogledajte odeljak Primer.

Imajte na umu da sVB pruža sintaksu za isti posao, postavljanjem događaja na Nothing, što se prevodi u poziv metode RemoveEventHandler kada se program kompajlira.

Dakle, ovaj kôd:

```
TextBox1.OnTextChanged = Nothing
```

je ekvivalentan (i biće kompajliran) ovom kodu:

```
TextBox1.RemoveEventHandler("OnTextChanged")
```

Nije bitno kako ste dodali handler za događaj OnTextChanged, jer će biti uklonjen bilo da ste ga dodali postavljanjem rukovaoce pomoću koda ili korišćenjem pod-profila koji prati konvenciju imenovanja (TextBox1\_OnTextChanged).

Imajte na umu da Nothing radi i sa starim Small Basic događajima kao što je Timer.Tick, koji se ne može koristiti sa metodom RemoveEventHandler:

```
Timer.Tick = Nothing
```

*Parametar:*

- *eventName*: Ime događaja za uklanjanje njegovog obrađivača. Ovo ime navodite kao string, ali ne morate da brinete o pogrešnom pisanju, jer će vam uređivač koda ponuditi listu za dovršavanje sa dostupnim imenima događaja.



**Primer:** Biblioteka Geometrics (koja je napisana pomoću sVB-a, a njen izvorni kôd postoji u folderu sa primerima) ima metod AllowDrag, koji prima kontrolu i registruje dva obrađivača za svoje događaje OnMouseMove i OnMouseDown, kako bi se korisniku omogućilo da je prevuče po formi.

```
Geometrics.AllowDrag(TextBox1)
```

Ne možete direktno pozvati metodu RemoveEventHandler iz vašeg projekta da biste uklonili ove obrađivače, ali biblioteka Geometrics vam pruža metodu PreventDrag, koja uklanja obrađivače događaja OnMouseDown za datu kontrolu kako bi onemogućila mogućnost prevlačenja.

```
Geometrics.PreventDrag(TextBox1)
```

Ovo je kôd koji se koristi za to, a možete ga pronaći u datoteci Global.sb u aplikaciji Geometrics u folderu sVB samples:

```
Sub AllowDrag(ciljnaKontrola) ' Dozvoljava korisniku da prevlači kontrolu mišem
targetControl.OnMouseMove = _OnMouseMove
targetControl.OnMouseDown = _OnMouseDown
EndSub

-----
Sub PreventDrag(ciljnaKontrola) ' Sprečava korisnika da prevlači kontrolu mišem
targetControl.RemoveEventHandler("OnMouseMove")
targetControl.RemoveEventHandler("OnMouseDown")
EndSub
```

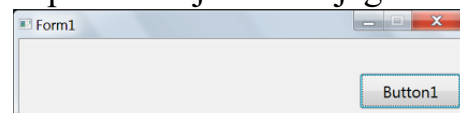
## Control.Right As Double

Dobija ili postavlja x-poziciju desne ivice kontrole u odnosu na levu ivicu njene roditeljske kontrole (forme). Ako koristite negativnu vrednost, kontrola će biti van forme.

**Primer:** Dodajte dugme na formu, dvaput kliknite na njega i napišite ovaj kôd u njegovom OnClick handler-u:

```
Button1.Right = Me.Width
```

Kada pokrenete program i kliknete na dugme, horizontalni položaj dugmeta će se promeniti tako da njegova desna ivica dodiruje desnu ivicu forme.



## Control.RightToLeft As Boolean

Dobija ili postavlja da li se sadržaj kontrole prikazuje zdesna nalevo. Ovo je korisno kada prikazujete jezike zdesna nalevo poput arapskog.

## Control.Rotate(ugao)

Rotira kontrolu za navedeni ugao.

**Parametar:**

- **ugao:** Ugao u stepenima za rotiranje oblika. Biće dodat trenutnoj vrednosti svojstva Control.Angle.

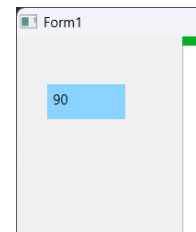
**Primer:** Dodajte ProgressBar i Labelu na formu i napišite ovaj kôd:

```
ProgressBar1.Rotate(120)
```



```
ProgressBar1.Rotate(-30)
Label1.Text = ProgressBar1.Angle
```

Kada pokrenete program, labela će prikazivati 90, i imaćete vertikalni ProgressBar ( $120 - 30 = 90^\circ$ ).



### Control.SendToBack()

Šalje trenutnu kontrolu na zadnji deo svih ostalih kontrola.

Ova metoda nema uticaja na forme.

Pogledajte primer dat za metodu BringToFront.

### Control.SetResourceDictionary(imeDatoteke)

Postavlja rečnik resursa kontrole tako što je učitava iz date datoteke. Rečnik resursa sadrži stilove za kontrolu i njene podređene kontrole. Ovo vam omogućava da dizajnirate napredne stilove pomoću drugih alata kao što je VS.NET, sačuvate ih u datoteku kao rečnik resursa, a zatim koristite ovu metodu da je učitate.

Ovo je napredna tema koja zahteva znanje o XAML-u i WPF-u, koje verovatno nemate, ali vam omogućava da koristite stilove i teme definisane za WPF, UWP ili WinUI3 (koje možete lako pronaći brзом pretragom na Google-u ili pitanjem nekoga ko ima znanje o ovim proizvodima). Ovo će vam pomoći da napravite lep dizajn, menjajući izgled, pa čak i rad kontrola!

#### *Napomene:*

- Ova metoda je najkorisnija kada se poziva iz forme, tako da stilovi utiču na sve ciljne tipove kontrola ako postoje na formi. U ovom slučaju, stilovi ne smeju imati ključeve (imena) da bi uticali na sve podređene kontrole ciljnih tipova.

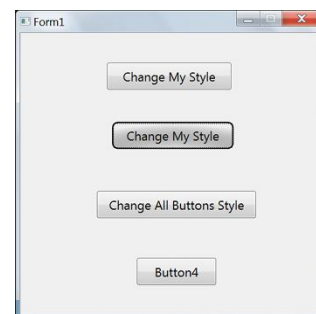
- U datoteci stilova, dodajte stilove ciljnim kontrolama, ali izbegavajte da ciljate prozor (formu), jer su kontrole zapravo postavljene na platno na formi, a mnoga svojstva prozora su podešena direktno na platno, a ne na formu. Dakle, nije lako napisati stil za promenu pozadine na formi (ne na platnu).

- Stilovi možda neće uticati na neka svojstva poput svojstva Button.Text, jer takvo svojstvo ne postoji u wpf-u, a sVB samo dodaje TextBlock u Button.Content. Dakle, koristite stilove da biste uradili ono što ne možete da uradite sa sVB kodom, a ostalo uradite sa kodom ako stilovi nisu uspešili. U suprotnom, možete koristiti stil da promenite šablon kontrole tako da možete naterati kontrolu da izgleda tačno onako kako želite.

- Ako datoteka stilova koristi slike, obavezno kopirajte te slike u direktorijum projekta i popravite putanje datoteka slika koje se koriste u datoteci stilova da budu relativne i da počinju sa ``, kao što je "sample.jpg", u suprotnom slika neće biti prepoznata u sVB-u.

#### *Parametar:*

- *imeDatoteke*: Datoteka koja sadrži rečnik resursa. Zapravo je to xaml datoteka, ali je bolje promeniti njenu ekstenziju u `.style` da se ne bi mešala sa datotekama dizajna forme koje imaju ekstenziju `.xaml`. Datoteke stilova treba da budu sačuvane u direktorijumu projekta i trebalo bi da koristite ime datoteke bez pune putanje (kao što je „rs.style“). Ne brinite o kopiranju datoteka stilova u fasciklu bin projekta, jer će sVB kopirati sve .xaml i .style datoteke u nju kada pokrenete program.



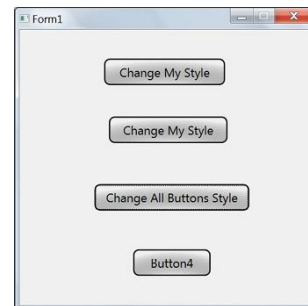
*Primeri:* Možete videti ovu metodu u akciji u projektu `Custom Styles` u fascikli samples.

Ovaj projekat sadrži datoteku stila pod nazivom 'RoundCorner.style', koja menja stil dugmeta da ima zaobljene ivice i menja mu boju pozadine. Možete primeniti ovaj stil na jedno dugme učitavanjem ove datoteke u njegov rečnik resursa. Upravo to radi i drugo dugme 'Change My Style' kada kliknete na njega:

```
Button3.SetResourceDictionary("RoundCorner.style")
```

Takođe možete primeniti ovaj stil na sva dugmad na formu odjednom učitavanjem ove datoteke u rečnik resursa obrasca. Ovo je upravo ono što dugme 'Change All Buttons Style' – *Promeni stil svih dugmadi* radi kada kliknete na njega:

```
Me.SetResourceDictionary("RoundCorner.style")
```



**Upozorenje:** Ako koristite stil koji menja šablon kontrole, ovo može uticati na neka od njegovih svojstava, tako da mogu prestati da rade. Na primer, u gornjem projektu, stil koji smo primenili na dugmad menja šablon dugmeta tako da koristi gradijentnu četkicu za crtanje pozadine, što potpuno ignoriše vrednost njegovog svojstva Background (koje koristimo u sVB pod nazivom BackColor). Dakle, nakon primene ovog stila, primenite korišćenje ovog koda koji neće imati uticaja na dugme1:

```
Button1.BackColor = Colors.Red
```

Dakle, trebalo bi pažljivo da kreirate stilove, da ne biste onemogućili svojstva kontrole, ili ako to uradite, dodajte sve željene efekte u stil, tako da ne morate da menjate izgled kontrole pomoću koda.

## Control.SetStyle(imeDatoteke, styleKey)

Postavlja stil kontrole tako što je učitava iz date datoteke.

Ovo vam omogućava da dizajnirate napredne stilove pomoću drugih alata kao što je VS.NET, sačuvate ih u datoteku kao rečnik resursa, a zatim koristite ovu metodu da biste je učitali.

Imajte na umu da je ova metoda slična metodi SetResourceDictionary, ali metod SetStyle ima drugi parametar koji prima ključ (ime) stila, pa je korisna kada želite da primenite stil samo na jednu kontrolu. Ali ako stil nema ime, možete pozvati SetResourceDictionary iz kontrole da biste primenili stil na nju, ali ovo takođe može primeniti druge stilove na kontrolu ako rečnik resursa ima mnogo stilova koji ciljaju isti tip kontrole.

Za više informacija, pogledajte napomene i primere metode SetResourceDictionary.

*Parametri:*

- *imeDatoteke*: Datoteka stila koja sadrži rečnik resursa.
- *styleKey*: Ključ stila. Rečnik resursa može da sadrži mnogo stilova koji ciljaju na istu kontrolu. Ova metoda će tražiti stil koji ima dati ključ i primeniti ga ako ga pronađe.

*Primeri:* Možete videti ovu metodu u akciji u projektu 'Custom Styles' u folderu samples. Ovaj projekat sadrži datoteku stila pod nazivom 'RoundCorner2.style', koja ima stil sa ključem 'RoundedButton'. Ovaj stil menja dugme da ima zaobljene ivice i menja mu boju pozadine. Možete primeniti ovaj stil na jedno dugme učitavanjem ove datoteke pomoću metode SetStyle. Upravo ovo radi prvo dugme „Change My Style“ – *Promeni moj stil* kada kliknete na njega:

```
Button1.SetStyle("RoundCorner2.style",  
"RoundedButton")  
Button1.ForeColor = Colors.Yellow
```



Imajte na umu da ako prvo kliknete na dugme „Change All Buttons Style“ – *Promeni stil svih*

*dugmadi*, Button1 će imati sivi stil, ali kada kliknete na Button1, dobiće zeleni stil. Ovo vam govori da poslednji stil koji primenite na kontrolu pobeđuje i nameće svoje vrednosti kontroli.

Ovo takođe važi kada promenite neka svojstva iz koda nakon što primenite stilove. Poslednja vrednost koju postavite svojstvu će prebrisati bilo koju staru vrednost.

Sada hajde da uradimo nešto zanimljivo: Šta mislite o prikazivanju sfernog dugmeta ovako?



Da biste to uradili, koristite dizajner obrazaca da promenite veličinu dugmeta Button1 na kvadrat 3x3 cm, zatim otvorite datoteku 'RoundCorner2.style' u beležnici i promenite vrednost svojstva Border.CornerRadius na 60:

```
<Border x:Name="border" CornerRadius="60"  
BorderBrush="Black" BorderThickness="2">
```

i sačuvajte datoteku. Sada kada pokrenete program i kliknete na Button1, ono će se pretvoriti u sferno dugme.

### Control.Tag As String

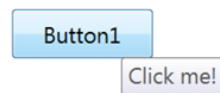
Ovo je dodatno svojstvo za čuvanje bilo koje vrednosti koju želite, a koja je povezana sa kontrolom. Možete sačuvati više vrednosti tako što ćete ih staviti u niz ili dinamički objekat i posediti ga na ovo svojstvo.

### Control.ToolTip As String

Dobija ili postavlja poruku koja će se prikazivati kao alatka kada zadržite pokazivač iznad kontrole.

*Primer:* Dodajte dugme na formu i koristite ovaj kôd:

```
Button1.ToolTip = "Klikni me!"
```



Pritisnite F5 da biste pokrenuli program i zadržite miš iznad dugmeta neko vreme (bez pomeranja miša). Dugme će prikazati savet alatke kao na slici:

### Control.Top As Double

Dobija ili postavlja y-poziciju gornje ivice kontrole u odnosu na gornju ivicu njene nadređene kontrole (forme). Ako koristite negativnu vrednost, gornja ivica kontrole će biti van forme, a ako vrh postane negativniji, cela kontrola se možda neće pojaviti na formi.

*Primer:* Dodajte dva dugmeta na formu i napišite ovaj kôd:

```
Button2.Top = -Button1.Height  
Sub Button1_OnClick()  
Button2.Top = Button2.Top + Button1.Height / 2  
EndSub
```

Kada pokrenete program, dugme2 se neće pojaviti na formi, jer je njegov vrh = - njegova visina. Kada kliknete na dugme1, vrh dugmeta2 će se povećati za polovinu svoje visine, tako da će se njegova polovina pojaviti na formi. Kada ponovo kliknete na dugme1, dugme2 će se u potpunosti pojaviti na formi. Možete nastaviti da klikćete na dugme button1 da biste povećali vertikalni položaj dugmeta button2, dok ne pređe donju ivicu forme i nestane.

## Control.TypeName

Dobija ime tipa kontrole, kao što su TextBox, Label i Button.

*Primer:* Dodajte različite tipove kontrola na formu i napišite ovaj kôd u globalnom području:

```
ForEach _Control In Me.Controls  
    TextWindow.WriteLine(_Control.TypeName)  
Next
```

```
Button  
Label  
ListBox  
TextBox  
Press any key to close the window...
```

Kada pokrenete program, TextWindow će prikazati imena kontrola<sup>1</sup>.

## Control.Validate( ) As Boolean

Primorava kontrolu da pokrene događaj OnLostFocus kako bi primenila bilo koju logiku validacije (potvrde) koju ste vi naveli u OnLostFocus rukovaocu, a zatim proverava svojstvo Error da bi se videlo da li kontrola ima greške ili ne. U većini slučajeva ne morate eksplicitno pozivati ovu metodu, jer je implicitno poziva metoda Form.Validate koja validira sve kontrole forme.

Ali možete je pozvati u nekim slučajevima, kao kada treba da prisilite validaciju kombinovanog okvira kada se izabrana stavka promeni. Upravo to se dešava u projektu Validation u folderu sa primerima:

```
Sub ComboBox1_OnSelection()  
    ComboBox1.Validate()  
EndSub
```

*Vraća:* True ako trenutna kontrola nema grešaka, ili False u suprotnom.

*Primer:* Možete validirati sve kontrole jednostavnim pozivanjem Me.Validate, ali ova metoda se zaustavlja ako je bilo koji unos kontrole nevažeći i ne validira sledeće kontrole dok korisnik ne ispravi grešku i ne pokuša ponovo. Možda ćete više voleti da validirate sve kontrole odjednom, a zatim pomerite fokus na prvu nevažeću, tako da korisnik vidi crvene okvire oko svih nevažećih kontrola i može da zadrži pokazivač miša iznad bilo koje od njih da bi video poruku o grešci u savetu alata. Ovo će mu omogućiti da ispravi sve greške odjednom. To možete učiniti tako što ćete napisati petlju za poziv metode Validate za svaku kontrolu na formi.

Ovaj kôd se koristi u dugmetu Validate na projektu Validation u folderu sa primerima:

```
IsValid = True  
ForEach control1 In Me.Controls  
    If control1.Validate() = False Then  
        Sound.PlayBellRing()  
        If IsValid Then ' Ovo je prva nevažeća kontrola  
            control1.Focus()  
            IsValid = False  
        EndIf  
    EndIf  
Next  
If IsValid Then ' Nisu pronađene greške. Obradi podatke  
    Me.ShowMessage("OK", "")  
EndIf
```

## Control.Visible As Boolean

Kada je True, kontrola se prikazuje na formi.

Kada je vrednost False, kontrola je skrivena.

*Primer:* Možete videti ovo svojstvo u akciji u projektu „CheckBox Sample“ u folderu sa

---

<sup>1</sup> Vidi i deo na kraju ovog dokumenta!

primerima. Koristi se u obrađivaču događaja `ChkShowText\_OnCheck` za prikazivanje ili skrivanje TextBox1, u skladu sa stanjem stanja polja za potvrdu:

```
Sub ChkShowText_OnCheck()  
state = ChkShowText.Checked  
If state Then ' Čeckirano  
    TextBox1.Visible = True  
    TextBox1.Enabled = True  
ElseIf state = "" Then ' Neodređeno  
    TextBox1.Visible = True  
    TextBox1.Enabled = False  
Else ' UNečekirano  
    TextBox1.Visible = False  
    TextBox1.Enabled = False  
EndIf  
EndSub
```

## Control.Width As Double

Dobija ili postavlja širinu kontrole.

Ako želite da se automatska širina prilagodi širini sadržaja kontrole, postavite ovo svojstvo na -1.

Automatsku širinu možete ograničiti i podešavanjem svojstva MaxWidth.

---

## Tip ControlTypes

Funkcioniše kao nabrojanje koje sadrži imena WinForms kontrola, kako bi vam bilo lako da proverite ime tipa kontrole koje vraća svojstvo **Control.TypeName**.

Tip ControlTypes ima sledeća svojstva, koja vraćaju ime odgovarajuće kontrole:

Button – *Dugme*  
CheckBox – *Polje za potvrdu*  
ComboBox – *Kombinovana kutija*  
Control – *Kontrola*  
DatePicker – *Birač datuma*  
Form – *Forma, Obrazac*  
Label – *Labela, Oznaka*  
ListBox – *Lista*  
MainMenu – *Glavni meni*  
MenuItem – *Stavka menija*  
ProgressBar – *Bar napretka*  
RadioButton – *Radio dugme*  
ScrollBar – *Traka za pomeranje*  
Slider – *Klizač*  
TextBox – *Tekstno polje*  
ToggleButton – *Dugme za prebacivanje*